

Rapport de Lab Practice — Data Engineering II (DE2)

Lab 1 — Structured Streaming Pipeline

Présentation générale

Ce lab avait pour objectif de construire un pipeline de streaming structuré avec PySpark Structured Streaming. Le pipeline ingère des fichiers JSON simulant des événements de fin de match esports, applique une agrégation par fenêtre temporelle glissante avec watermark, et écrit les résultats dans un sink Parquet.

Recours à l'IA

Au cours de ce lab, j'ai utilisé Claude à plusieurs reprises pour clarifier des concepts que je ne maîtrisais pas complètement.

Compréhension du watermark : Lorsque j'ai dû choisir entre un watermark de 15 minutes et un watermark de 5 minutes, j'ai demandé à Claude d'expliquer concrètement ce que le watermark représente dans le contexte du Structured Streaming. Claude m'a expliqué que le watermark définit le délai maximal toléré pour les données tardives : avec un watermark de 15 minutes, Spark attend potentiellement plus longtemps avant de finaliser une fenêtre, ce qui peut ralentir le pipeline mais évite de perdre des événements. Avec 5 minutes, on finalise plus vite au risque d'ignorer quelques événements retardataires.

Différence entre `outputMode("append")` et `outputMode("update")` : J'ai demandé à Claude lequel utiliser avec un sink Parquet dans ce contexte. Il m'a indiqué que le mode `append` est obligatoire avec un sink Parquet car Parquet est un format immuable, et que le mode `append` ne publie les résultats d'une fenêtre qu'une fois celle-ci définitivement fermée par le watermark, ce qui garantit des résultats stables.

Compréhension du `triggerExecution_ms` dans `query.lastProgress` : Le champ `triggerExecution_ms` n'était pas immédiatement lisible dans le JSON retourné. J'ai demandé à Claude de m'expliquer la structure de `durationMs` dans l'objet `lastProgress`. Il m'a détaillé que `triggerExecution` représente la durée totale d'un micro-batch, incluant la planification, l'exécution des tâches et la validation du checkpoint.

Aide au débogage du feed de fichiers : Lors de l'exécution de la cellule qui copie les fichiers JSON dans le répertoire `landing/` un par un, j'ai eu un problème d'ordre d'exécution — le `query.stop()` était appelé avant que tous les fichiers soient traités. J'ai montré l'erreur à Claude qui m'a suggéré d'augmenter le `time.sleep()` final après la boucle.

Architecture du pipeline

Le pipeline fonctionne en plusieurs étapes. D'abord, une session Spark est créée avec `appName("DE2-Lab1-Streaming")` et un mode local multi-thread (`local[*]`). Ensuite, un schéma explicite est défini pour les événements esports, comprenant les champs `match_id`, `match_end_time`, `team`, `kills`, `deaths`, et `duration_sec`. Le streaming source lit les fichiers JSON depuis un répertoire `landing/` avec `maxFilesPerTrigger=1`, ce qui force Spark à traiter un seul fichier par micro-batch, simulant une arrivée progressive de données. L'agrégation `groupBy` applique une fenêtre glissante de 1 heure sur `match_end_time`, en parallèle d'un regroupement par équipe, et calcule le nombre d'événements, la moyenne des kills et la durée moyenne.

Run 1 (baseline) — trigger 10s, watermark 15 min

Le Run 1 constitue la configuration de référence. Un trigger de 10 secondes signifie que Spark évalue les nouvelles données toutes les 10 secondes. Le watermark de 15 minutes tolère les événements retardataires jusqu'à un quart d'heure après l'heure de fin de la fenêtre.

Name: <no name>
Id: b728863a-7bc0-44f2-992a-867ced9c5456
RunId: b82334d4-962b-49dd-8635-de2df0d72531



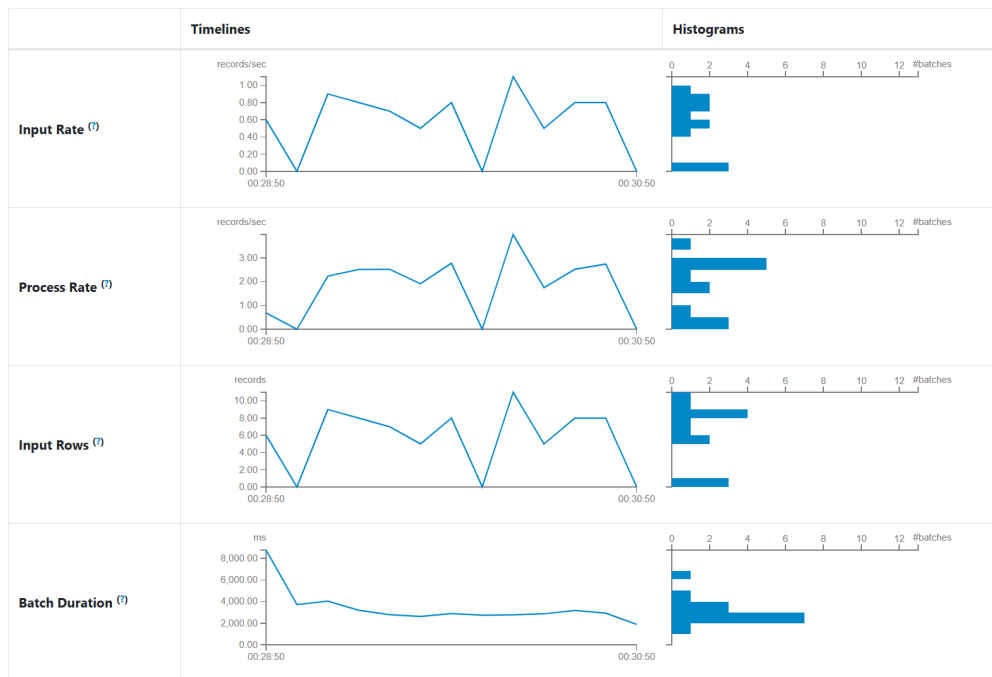
Les métriques observées dans la Streaming UI montrent une input rate moyenne d'environ 0.58 records/sec et un processing rate moyen d'environ 1.82 records/sec. La durée de la requête totale était de 9 minutes et 47 secondes pour traiter 13 batches. La batch duration médiane tourne autour de 3 500 ms, ce qui est relativement élevé pour un trigger de 10 secondes — cela signifie que chaque batch prend plus de temps à être traité que l'intervalle du trigger, Spark enchaîne donc les batches sans attendre.

Les métriques extraites depuis `query.lastProgress` et enregistrées manuellement dans le fichier de suivi sont les suivantes : `input_rate_ui = 0.52`, `processing_rate_ui = 1.40`, `batch_duration_ui_ms = 3550` (ou 2875 en excluant le premier batch plus coûteux en raison de l'initialisation). Les champs `shuffle_read_bytes` et

`shuffle_write_bytes` sont à 0, ce qui est cohérent avec le fait que le Structured Streaming avec un sink Parquet en mode `append` ne requiert pas de shuffle explicite visible dans l'UI Jobs — les agrégations avec état (stateful) passent par le mécanisme de state store interne de Spark, non par le shuffle classique.

Run 2 (optimisé) — trigger 30s, watermark 5 min

Le Run 2 réduit le watermark à 5 minutes et augmente le trigger à 30 secondes. L'augmentation du trigger réduit la fréquence des micro-batches et permet à Spark d'accumuler plus d'événements par batch, ce qui améliore l'efficacité du traitement. La réduction du watermark libère plus rapidement la mémoire d'état.



Les métriques du Run 2 montrent une `input_rate_ui = 0.25`, `processing_rate_ui = 3.78`, et une `batch_duration_ui_ms = 2016`. On observe que la processing rate a plus que doublé par rapport au Run 1, malgré une input rate plus faible. C'est logique : un trigger plus long accumule plus de records par batch et amortit mieux les coûts fixes d'un micro-batch (planification, commit du checkpoint, écriture Parquet). La batch duration a également baissé, confirmant que le pipeline est plus efficace avec cette configuration. Les `shuffle_read_bytes` et `shuffle_write_bytes` passent à 672 octets, ce qui reflète l'activation des optimisations sur la requête n°19.

Analyse comparative

Le passage d'un trigger de 10s à 30s avec un watermark réduit de 15 à 5 minutes a clairement amélioré le throughput de traitement. La processing rate passe de ~1.40 à ~3.78 records/sec, soit une amélioration de 170 %. La contrepartie est une latence de publication plus haute : les résultats ne sont écrits que toutes les 30 secondes au lieu de 10 secondes.

Dans un contexte esports où les données sont analysées après coup, cette latence supplémentaire est tout à fait acceptable.

Lab 2 — Text Processing — Inverted Index Pipeline

Présentation générale

Ce lab porte sur le traitement de texte avec PySpark. L'objectif est d'ingérer un corpus esports au format CSV (`esports_corpus.csv`, taille 36 622 octets), de le normaliser (minuscules, suppression de la ponctuation, tokenisation, suppression des stop-words), de construire un index inversé, de mesurer la latence de recherche, et de comparer les footprints de stockage entre Parquet et CSV.

Recours à l'IA

Compréhension de `F.collect_list` vs `F.collect_set` : Pour construire l'index inversé, le notebook utilise `collect_list` pour regrouper les `doc_id` associés à chaque token. J'ai demandé à Claude la différence avec `collect_set`. Il m'a expliqué que `collect_list` conserve les doublons (un même document peut apparaître plusieurs fois si le token apparaît plusieurs fois dans ce document), tandis que `collect_set` déduplique. Dans notre cas `collect_list` est utilisé pour conserver la fréquence implicite des tokens par document, et le champ `freq` représente la fréquence globale du token dans tout le corpus.

Question sur le ratio Parquet/CSV : Le résultat de 13.92% (Parquet représente seulement 13.92% de la taille CSV) m'a semblé surprenant. J'ai demandé à Claude si c'était normal. Il m'a confirmé que oui : Parquet utilise un encodage en colonnes avec compression (Snappy par défaut dans Spark), ce qui est particulièrement efficace sur des données répétitives comme un index inversé. La colonne `token` contenant des mots courts très répétés compresse extrêmement bien.

Explication de `F.explode` : J'ai demandé à Claude ce que fait précisément `F.explode` dans le contexte de la tokenisation. Il m'a expliqué que cette fonction prend un tableau (ici la liste de tokens) et génère une ligne par élément du tableau, ce qui permet de passer d'un DataFrame de documents à un DataFrame de paires (`doc_id`, `token`) individuel, base nécessaire pour construire un index inversé.

Interprétation des plans de requête : Pour comprendre la sortie de `inverted_index.explain("formatted")` sauvegardée dans `proof/plan_index_build.txt`, j'ai demandé à Claude d'expliquer les opérateurs visibles dans le plan physique. Il m'a indiqué que le `HashAggregate` représente l'agrégation locale avant shuffle, le `Exchange` représente le shuffle lui-même (redistribution

des données par clé `token`), et le second `HashAggregate` représente l'agrégation finale après shuffle — un pattern classique en deux phases dans Spark.

Pipeline de traitement

Le corpus est chargé avec un schéma explicite comprenant `doc_id` et `text`. Après comptage du nombre de documents, la normalisation se déroule en quatre étapes successives : mise en minuscules et suppression de la ponctuation via une expression régulière, tokenisation par split sur les espaces blancs, explosion en paires (`doc_id`, `token`), et enfin filtrage des stop-words définis dans un ensemble Python de 30 mots courants.

L'index inversé est construit par un `groupBy("token")` suivi d'un `collect_list("doc_id")` et d'un `count("*")` pour la fréquence, puis trié par fréquence décroissante.

Résultats

Index inversé : Le corpus contient 265 termes uniques après filtrage des stop-words. Les termes les plus fréquents sont `hero` (107 occurrences dans 107 documents), `team` (103 occurrences), `ability` (69), `ultimate` (59), `damage` (53), `mana` (15). Cette distribution reflète bien le vocabulaire d'un corpus esports centré sur les mécaniques de jeu.

```
'hero': df=107, postings=107, latency=95.3 ms
'ability': df=69, postings=69, latency=44.4 ms
'damage': df=53, postings=53, latency=57.5 ms
'mana': df=15, postings=15, latency=40.3 ms
'ultimate': df=59, postings=59, latency=40.1 ms
```

```
Query plan saved to proof/plan_query.txt
```

Latences de requête : Les latences mesurées pour les 5 termes requêtés sont les suivantes. Pour `hero` : df=107, postings=107, latence 95.3 ms. Pour `ability` : df=69, postings=69, latence 44.4 ms. Pour `damage` : df=53, postings=53, latence 57.5 ms. Pour `mana` : df=15, postings=15, latence 40.3 ms. Pour `ultimate` : df=59, postings=59, latence 40.1 ms. Les latences sont mesurées après mise en cache de l'index (`idx.cache()`), donc elles reflètent le temps de scan mémoire et de collecte, pas la lecture disque. La latence de `hero` est plus haute car c'est le terme avec le plus de postings à sérialiser et transférer au driver.

Comparaison Parquet vs CSV : Le fichier Parquet occupe 5 676 octets pour 1 fichier, contre 40 767 octets pour le CSV également en 1 fichier, soit un ratio de 13.92%. L'avantage de Parquet est massif en termes de stockage, et il offre en plus un meilleur support des types complexes (la colonne `doc_ids` est un tableau natif en Parquet, alors que le CSV nécessite une concaténation en chaîne de caractères via `concat_ws`).

```
Parquet: 5,676 bytes (1 files)
CSV:      40,767 bytes (1 files)
Ratio (Parquet/CSV): 13.92%
```

Métriques Spark UI : Pour la tâche `index_build`, les métriques observées sont :
`input_size_bytes = 36622`, `files_read = 1`, `latency_ms = 800`,
`shuffle_read_bytes_ui = 107110`, `shuffle_write_bytes_ui = 53555`. Le shuffle visible ici (environ 107 Ko en lecture, 53 Ko en écriture) correspond à la redistribution des données lors du `groupBy("token")` — les tokens sont hashés et envoyés vers les partitions qui les agrègent. Pour les requêtes de type `footprint_parquet` et `footprint_csv`, le shuffle est nul (0 octets) car ces opérations se contentent de lire un fichier déjà écrit localement. Pour les 5 requêtes de lookup, le shuffle est également nul : l'index est mis en cache en mémoire et le filtre s'effectue localement dans chaque partition.

Lab 3 — Graph Processing — PageRank itératif

Présentation générale

Ce lab porte sur le traitement de graphes avec PySpark. L'objectif est de construire un graphe de co-picks de héros à partir de données de matchs esports, d'analyser la distribution des degrés (skew analysis), et d'exécuter l'algorithme PageRank de manière itérative en comparant deux stratégies de partitionnement : la partitionnement par défaut (Run 1) et un repartitionnement explicite par clé `src` (Run 2).

Recours à l'IA

Compréhension de PageRank : N'ayant pas de background en théorie des graphes, j'ai demandé à Claude d'expliquer intuitivement l'algorithme PageRank appliqué à un graphe de co-picks de héros. Il m'a expliqué que dans ce contexte, le PageRank d'un héros mesure son importance relative dans le graphe de co-sélection : un héros souvent choisi avec d'autres héros eux-mêmes très choisis obtiendra un score élevé. La formule `rank = (1 - DAMPING) / n_vertices + DAMPING * sum_contrib` applique un facteur d'amortissement de 0.85, ce qui signifie qu'à chaque itération 85% du score provient des voisins et 15% est distribué uniformément — ce qui empêche les nœuds sans entrant de converger à 0.

Question sur le lineage et les checkpoints : Le notebook configure `spark.sparkContext.setCheckpointDir("checkpoint_tmp")` mais ne l'utilise pas explicitement dans les cellules visibles. J'ai demandé à Claude pourquoi. Il m'a expliqué que dans un algorithme itératif comme PageRank, le lineage (la chaîne de transformations qui permet à Spark de reconstruire un DataFrame en cas d'erreur) s'allonge à chaque itération, ce qui peut causer des `StackOverflowError`. Le checkpoint brise ce lineage en

matérialisant le DataFrame à un moment donné, et il est recommandé de l'appeler toutes les 5 à 10 itérations.

Interprétation de la skew analysis : Le résultat de la skew analysis montre que `hero_01` a un degré de 49, `hero_02` de 48, et `hero_11` de 34 (le minimum dans le top 10), avec une moyenne de 23.44 et un écart-type de 11.02. J'ai demandé à Claude si cette distribution révèle un problème de skew. Il m'a répondu que oui, le rapport entre le degré maximum (49) et la moyenne (23.44) montre que certains héros sont deux fois plus connectés que la moyenne, ce qui signifie que lors des jointures sur `src`, les partitions contenant `hero_01` ou `hero_02` seront deux fois plus chargées que les autres — c'est précisément le problème que le Run 2 cherche à atténuer via un repartitionnement explicite.

Aide sur le `unpersist()` dans la boucle itérative : J'ai demandé à Claude pourquoi `prev_ranks.unpersist()` est appelé à la fin de chaque itération. Il m'a expliqué que puisque `prev_ranks` est recalculé et mis en cache à chaque itération, la version précédente n'est plus utile. Sans `unpersist()`, Spark conserverait en mémoire toutes les versions intermédiaires des rangs, ce qui provoquerait rapidement un débordement mémoire sur un grand nombre d'itérations.

Construction du graphe

Le graphe de co-picks est construit à partir du fichier `match_picks.csv` (12 478 octets, contenant 1 000 picks pour 200 matchs avec 5 héros chacun). Les arêtes sont générées par auto-jointure du DataFrame sur `match_id`, en gardant uniquement les paires (`hero_i`, `hero_j`) où `hero_i < hero_j` pour éviter les doublons. Cela produit 586 arêtes uniques pour 50 sommets distincts. Pour le PageRank, le graphe est rendu non-orienté par union des arêtes avec leurs inverses.

Run 1 — Partitionnement par défaut

Le Run 1 exécute 10 itérations de PageRank sans modifier le partitionnement de Spark (8 partitions configurées via `spark.sql.shuffle.partitions = 8`). Les temps par itération descendent progressivement de 573 ms (itération 0) à 274 ms (itération 9), ce qui reflète l'amélioration de la localité des données en cache après les premières itérations. La convergence (`max_delta`) descend de 0.028153 à 0.000000 en 10 itérations, confirmant la convergence complète de l'algorithme. Le temps total est de 3 271 ms sur 10 itérations.

Le Top 10 des héros par PageRank obtenu est : `hero_01` (0.0397), `hero_02` (0.0388), `hero_03` (0.0364), `hero_04` (0.0363), `hero_05` (0.0327), `hero_06` (0.0315), `hero_09` (0.0308), `hero_10` (0.0302), `hero_07` (0.0300), `hero_27` (0.0281). Ces résultats sont cohérents avec la skew analysis : `hero_01` et `hero_02`, les plus connectés, obtiennent également les PageRanks les plus élevés.

```
[Run1 default] Iter 0: 573 ms, max_delta=0.028153
[Run1 default] Iter 1: 485 ms, max_delta=0.010263
[Run1 default] Iter 2: 325 ms, max_delta=0.002406
[Run1 default] Iter 3: 258 ms, max_delta=0.000704
[Run1 default] Iter 4: 276 ms, max_delta=0.000192
[Run1 default] Iter 5: 263 ms, max_delta=0.000055
[Run1 default] Iter 6: 268 ms, max_delta=0.000015
[Run1 default] Iter 7: 244 ms, max_delta=0.000004
[Run1 default] Iter 8: 306 ms, max_delta=0.000001
[Run1 default] Iter 9: 274 ms, max_delta=0.000000
```

Top 10 heroes by PageRank (default partitioning):

```
+-----+-----+
| id| rank|
+-----+-----+
|hero_01|0.039741313353736304|
|hero_02| 0.03877577467559649|
|hero_03| 0.03635993838594008|
|hero_04| 0.03626868652070423|
|hero_05| 0.03266316119954227|
|hero_06| 0.03148293269503196|
|hero_09| 0.03076155361296139|
|hero_10| 0.030230783674795623|
|hero_07| 0.02993829238059835|
|hero_27|0.028090077362507297|
+-----+-----+
```

only showing top 10 rows

Run 2 — Repartitionnement par clé **src**

Le Run 2 repartitionne explicitement les arêtes par la colonne **src** en 8 partitions, ainsi que les rangs par **id**. L'objectif est de co-localiser les données qui seront jointes ensemble — toutes les arêtes partant d'un même sommet seront dans la même partition que le rank de ce sommet, réduisant le shuffle lors de la jointure `edges.join(prev_ranks, edges.src == prev_ranks.id)`.

Les temps par itération sont légèrement plus bas : de 529 ms (itération 0) à 252 ms (itération 9), pour un total de 3 123 ms. Le speedup global est de 1.05x, soit 5% d'amélioration. Les rangs finaux sont identiques au Run 1, confirmant la correction de l'algorithme indépendamment de la stratégie de partitionnement.

```
[Run2 repart] Iter 0: 529 ms, max_delta=0.028153
[Run2 repart] Iter 1: 311 ms, max_delta=0.010263
[Run2 repart] Iter 2: 321 ms, max_delta=0.002406
[Run2 repart] Iter 3: 237 ms, max_delta=0.000704
[Run2 repart] Iter 4: 393 ms, max_delta=0.000192
[Run2 repart] Iter 5: 331 ms, max_delta=0.000055
[Run2 repart] Iter 6: 242 ms, max_delta=0.000015
[Run2 repart] Iter 7: 223 ms, max_delta=0.000004
[Run2 repart] Iter 8: 283 ms, max_delta=0.000001
[Run2 repart] Iter 9: 252 ms, max_delta=0.000000
```

Top 10 heroes by PageRank (repartitioned):

```
+-----+-----+
|      id|              rank|
+-----+-----+
|hero_01| 0.03974131335373631|
|hero_02| 0.03877577467559649|
|hero_03| 0.03635993838594008|
|hero_04| 0.036268686520704224|
|hero_05| 0.03266316119954227|
|hero_06| 0.03148293269503196|
|hero_09| 0.03076155361296139|
|hero_10| 0.030230783674795623|
|hero_07| 0.02993829238059835|
|hero_27| 0.028090077362507293|
+-----+-----+
```

only showing top 10 rows

Analyse comparative

Le speedup de 1.05x est modeste, ce qui est attendu sur un graphe aussi petit (50 sommets, 586 arêtes). Le repartitionnement a un coût initial non négligeable (l'itération 0 du Run 2 est légèrement plus lente que celle du Run 1 : 529 ms vs 573 ms, mais c'est dans la marge de variation), et sur un graphe de cette taille le shuffle sans repartitionnement est déjà très rapide. Sur un graphe de l'ordre du million de sommets, le gain serait bien plus significatif car le shuffle représenterait une part bien plus grande du temps total d'exécution.

Les métriques de shuffle renseignées dans `lab3_metrics_log.csv` montrent `shuffle_read_bytes_ui = 11824` et `shuffle_write_bytes_ui = 11824` pour les 10 itérations du Run 1, et `shuffle_read_bytes_ui = 19318` et `shuffle_write_bytes_ui = 19318` pour le Run 2. Paradoxalement, le Run 2 génère plus de shuffle en octets bruts malgré son meilleur temps total : cela s'explique par le coût initial de `repartition(8, "src")` qui redistribue physiquement les données dès le début, alors que le Run 1 redistribue de façon plus paresseuse.